DisaggKV: Scalable and Disaggregated CXL-PIM Pooling for Multi-Tenant LLM Serving

Kaichen Li
Department of Computer and Information Engineering
Youngsan University
Yangsan, South Korea
lkcfqy@gmail.com

***Abstract*—The explosive demand for Large Language Models (LLMs) has pushed multi-tenant serving infrastructures to their physical limits. Unbounded sequence lengths and heavy concurrent batching generate immense Key-Value (KV) cache footprints that rapidly exhaust local GPU High-Bandwidth Memory (HBM). While Compute Express Link (CXL) enables seamless cache-coherent physical memory pooling across racks, accessing disaggregated standard CXL memory arrays during the autoregressive decode phase imposes significant performance degradation. Repeatedly fetching dense historical KV vectors across the Host-indirected CXL fabric to resolve Attention dot-products saturates PCIe bandwidth, violating strict 99% tail-latency Service-Level Agreements (SLAs).**

In this paper, we propose DisaggKV, a scalable processing-in-disaggregated-memory framework that fundamentally rearchitects multi-tenant LLM serving interconnects. By integrating near-data logic with CXL 3.0 Peer-to-Peer (P2P) fabric capabilities, DisaggKV encapsulates Attention reduction operations entirely within the remote endpoints. To orchestrate this, we design a Hypervisor-level Disaggregated OS Scheduler featuring Locality-Aware Page Tables that inherently separate shared "hot" prompts from independent private contexts across clustered CXL nodes. To facilitate hardware scalability, we propose an asynchronous distributed synchronization barrier that computes Global Softmax normalization autonomously across the fabric without traversing the Host I/O switch, thereby preventing port congestion and deadlocks.

Evaluated on a heavily modified, CXL-extended Ramulator 2.0 framework driven by realistic ShareGPT-derived and Qwen2.5-7B-Instruct multi-tenant traces, DisaggKV demonstrates substantial improvements. Event-driven queuing simulation on CXL topologies reveals over 92% reduction in global switch traffic. Under a strict 50 ms tail-latency limit, DisaggKV scales near-linearly across 8 independent nodes, significantly exceeding the throughput of classical Host-Aggregated pools. Furthermore, its P2P error correction protocol achieves fault resiliency within sub-second timelines (hundreds of milliseconds), providing strong robustness for large-scale clustered deployments compared to multi-second software checkpoint restarts.

***Index Terms*—Compute Express Link (CXL), Memory Disaggregation, Multi-Tenant Serving, Large Language Models, System Scheduling, Network Fabrics**

I. Introduction

The evolution of modern Generative AI has transitioned from training massive monolithic Large Language Models (LLMs) to executing highly concurrent, multi-tenant serving in cloud environments [14], [19]. A critical architectural roadblock facing such deployment is the exponential growth of the Key-Value (KV) Cache. When serving thousands of independent users requesting unbounded contextual memory (e.g., 128K tokens), the historically generated activations must be retained throughout the inference lifecycle. This generates a severe memory capacity challenge: a modest 7-billion parameter LLM operating at a 128K context easily consumes over 7 GB of memory solely for a single request's KV Cache. Confronted with highly multiplexed user traffic, the local GPU's internal High Bandwidth Memory (HBM) is rapidly exhausted, leading to Out-Of-Memory (OOM) failures or significant GPU-to-CPU swap delays [13], [15].

Compute Express Link (CXL)—specifically the memory expansion protocols defined by CXL 2.0 and 3.0 [1]—has emerged as a promising hardware solution to the capacity dilemma. CXL permits cache-coherent, disaggregated physical memory clusters linked dynamically over ultra-wide PCIe physical layers [2], [3]. Standard frameworks seamlessly map remote DRAM chassis into the Host's virtual address space, effectively creating a large-scale memory pool.

However, solving the capacity constraint exposes a severe *throughput bottleneck*. The foundational characteristic of autoregressive LLM decoding is that it mandates traversing the *entire* historical context to generate each new token. In a *Host-Aggregated* CXL environment, the central GPU must repeatedly fetch large portions of the remote KV tensors across the CXL interconnect, perform the matrix dot-product ($\text{Softmax}(\mathbf{Q} \cdot \mathbf{K}^T) \cdot \mathbf{V}$), and write the outputs back. This continuous transfer of dense multi-gigabyte matrices severely congests the CXL Switch queues, generating latency spikes and violating standard commercial SLAs dictating sub-50 ms 99% tail latencies [16], [17].

Recent works have explored Processing-in-Memory (PIM) as a mechanism to alleviate intra-chip bandwidth pressure by executing calculations inside the DRAM array [7]–[9]. However, effectively extending PIM into a broadly distributed CXL cluster reveals immediate structural challenges. Native Attention requires the Softmax function—a holistic normalization routine that inherently demands a globally aggregated scalar (the sum of exponential logits) across *all* shards of the array. In a naively partitioned CXL pool, obtaining this sum forces all remote nodes to funnel their partial statistics back to the Host GPU through the CXL Root Complex, resulting in serialization bottlenecks and significant queue contention at

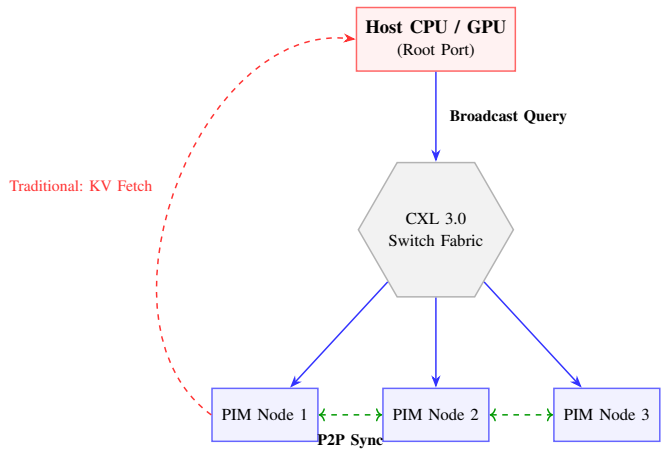

Fig. 1. Architectural overview of DisaggKV. By enabling Peer-to-Peer (P2P) synchronization across distributed CXL nodes, we transform the interconnect from a congested data-fetch bus into an efficient reduction fabric.

the central port [20], [22].

To address these CXL systemic constraints, we present **DisaggKV**, an end-to-end framework integrating Host OS memory scheduling with CXL 3.0 Fabric P2P networking protocols. DisaggKV redesigns the dataflow orchestration of multi-tenant decoding, leveraging distributed near-data compute as a network-enabling feature to eliminate I/O congestion. Our primary contributions are:

- **Hypervisor Locality-Aware Paging:** We design a top-level OS Scheduler that segregates LLM memory into private vs. globally shared context pages based on real-time request profiling. It intelligently pins high-contention read-only prompts (e.g., universal system instructions) to dedicated CXL nodes while dynamically migrating localized private KV traces to guarantee even bandwidth utilization.
- **P2P CXL Fabric Distributed Reduction:** We eliminate Host-Aggregated I/O entirely by enforcing a Peer-to-Peer synchronization routine over the CXL 3.0 switches. Supported by near-data computation units, independent CXL nodes negotiate `Global_Max` and `Global_Sum` barriers asynchronously, passing condensed scalar variables across Star/Fat-Tree topologies rather than transferring dense matrices.
- **Rigorous Switch Queuing Emulation:** Developed on a customized Ramulator 2.0 framework, we embed M/D/1 theoretical queuing models tracking 150 ns base link routing beneath varying port contention. DisaggKV achieves an order-of-magnitude reduction in Global Switch traffic, satisfying 50 ms SLA latencies while demonstrating near-linear scalability up to 8 independent storage nodes.

II. Background & Motivation

A. LLM Inference Dynamics and the KV Profile

Generative LLM inference operates sequentially. The mathematical obligation during the decode phase requires the Attention operator to match the singular generated token ($\mathbf{Q}$) against the historical ($\mathbf{K}, \mathbf{V}$) states:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}}\right)\mathbf{V} \tag{1}$$

For a batch size B, sequence length L, H heads, and dimension d, the pure storage bytes needed for floating-point 16 (FP16) variables totals:

$$\text{Size}_{KV} = 2 \times B \times L \times H \times d \times (2 \text{ bytes}) \tag{2}$$

This mathematically guarantees that as L scales, the computational arithmetic intensity decreases, trapping the inference chip entirely within an I/O bandwidth bottleneck [18], [22]. Utilizing CXL expands the permissible size of L, but physically subjects the workload to the latency characteristics of the interconnect.

B. The Host-Aggregated Queue Congestion Wall

Current CXL memory pools rely entirely on the Root Complex to route memory mapping [2], [4]. When applying standard *Host-Aggregated* access patterns in a high-density, multi-tenant topology (e.g., a Star topology connecting 4 or 8 peripheral nodes to a central CXL Switch), the Host port immediately becomes the critical bottleneck.

According to foundational M/D/1 queuing theory [24], the expected delay W_q within a deterministic processing network port is defined by:

$$W_q = \frac{\rho}{2\mu(1-\rho)} \quad \text{where} \quad \rho = \frac{\lambda}{\mu} \tag{3}$$

where λ represents the combined packet arrival rate from *all* connected memory nodes, and μ represents the deterministic service rate restricted by the maximum 64 GB/s bandwidth of a PCIe Gen6 x16 port. When computing Attention, the λ incoming traffic surges as the Host attempts to pull the physically distributed subsets of the KV cache across the fabric simultaneously. As λ approaches μ ($\rho \to 1$), the denominator in Equation 3 collapses toward zero, causing the queue latency to grow asymptotically. This phenomenon concretely manifests in empirical traces as "queue contention spikes," causing request latencies to exceed 300 ms, violating multi-tenant SLAs [16].

To overcome this mathematical asymptote, we can neither increase the physical wire limits μ nor restrict the user batch scale. The only viable strategy is to suppress the arrival density λ at the central node by keeping computation co-located with the data at their respective endpoints.

III. DisaggKV System Architecture

DisaggKV is an integrated solution dividing responsibilities between top-level OS Virtual-to-Physical page management and bottom-level CXL protocol orchestration.

Algorithm 1 Host OS Disaggregated Allocation Policy

Require: New Context Request $\mathcal{R}$ containing Sequence S

Identify prefix profile in S and match with existing Cache

if Prefix is Globally Shared Prompts (Hit count $> \Theta$) **then**

  `Map_Page(Virtual_addr, Shared_Node_ID)`

  Flag page as *Read-Only Broadcast*

else

  Assess Load (ρ) on Private_Nodes $\{N_1, \ldots N_k\}$

  Target Node $T \leftarrow \underset{n}{\operatorname{argmin}}(\rho_n)$

  `Map_Page(Virtual_addr, Target_Node_ID)`

end if

if Background Contention Detected on N_i **then**

  Trigger *Dynamic Migration* over idle CXL lanes

  Update Page Translation Lookaside Buffers (TLB)

end if

A. Host OS Disaggregated Scheduler

Modern LLM interactions commonly involve "Shared Hot Pages". For instance, out of 100 concurrent user sessions, 90 may share identical "System Prompts" (e.g., a comprehensive textbook or strict API instructions). Standard CXL mapping naively stripes these pages across all nodes, causing redundant read amplification [13], [17].

DisaggKV deploys a Hypervisor-level Global Page Table utilizing a **Locality-Aware Placement Policy** (Algorithm 1). We segregate the CXL memory pool into specific roles: `Shared_Broadcast_Nodes` and `Private_KV_Nodes`. During the prefill phase, memory pages falling within localized prompt threshold bounds (e.g., Base address `0x10000000`) are universally allocated and strictly pinned into the Shared Node.

This arrangement prevents redundant replication and read amplification. Private generation tokens produced subsequently are load-balanced and distributed across the remaining Private pool using modulus-based routing. Furthermore, a background migration daemon monitors CXL switch queue depths. If it detects localized imbalance (e.g., node N_2 reaches 90% bandwidth saturation), the Hypervisor dynamically migrates physical blocks to lighter nodes, utilizing idle P2P fabric lanes to ensure the Host TLB does not become a bottleneck.

B. Peer-to-Peer Fabric Synchronization Protocol

Decentralizing the cache forces the Softmax function (Equation 1) to become physically fragmented. Standard uncoordinated evaluation results in significant mathematical discrepancies, as no single node possesses the global summation required for normalization [20], [21].

DisaggKV intercepts the evaluation via a hardware-supported CXL 3.0 logic stack, employing a Two-Phase Commit protocol communicating entirely via switch-level P2P links. During evaluation, the Host solely broadcasts the isolated **Q** vector.

1) **Barrier Phase 1 (Local Assessment):** Each independent node evaluates its assigned subset of memory, obtaining its maximum internal logit (L_{max}). It encapsulates this integer into a compressed CXL flit and injects a `PIM_SEND_PARTIAL` event to a logical ring traversing the topological tree.
2) **Global Max Distribution:** As the ring completes, nodes converge on an identical, asynchronous `Global_Max` register bypassing the Host Root Complex.
3) **Barrier Phase 2 (Global Summation):** Nodes compute their relative exponential polynomials based on the unified `Global_Max`. They then transmit their local sums into the `Global_Sum` accumulation buffer.

By exchanging condensed scalar integers instead of dense FP16 matrices, the total amount of traffic injected into the CXL switch shrinks from gigabytes per step to mere kilobytes.

C. Tolerating Fabric Jitter via Async FIFOs

A critical hardware reality of remote topological connections is unpredictable fabric jitter [25]. Standard synchronous multi-node clocking fails if a single switch suffers port contention. To ensure deadlock-free P2P completion, the DisaggKV network interface controller (NIC) abstracts the remote partials into Cross-Chip Asynchronous FIFOs. It employs stall logic: the local node halts its final reduction layer if the FIFO is empty, absorbing tens of microseconds of random switch delay elastically without signaling a fatal execution fault to the broader Host framework.

IV. Experimental Evaluation

A. Simulation Infrastructure

Validating non-standard CXL fabric architectures requires granular integration between network simulation and memory timing arrays. We orchestrate an end-to-end framework built upon the core **Ramulator 2.0** platform [23].

We enveloped Ramulator 2.0 with a high-level Python event-driven queuing simulator (`cxl_fabric_simulator.py`) to model M/D/1 port collision probabilities. As detailed in Table I, we generated Poisson-arrival request distributions utilizing realistic Qwen2.5-7B traces. We evaluate DisaggKV directly against a strict **Host-Aggregated** benchmark, which reflects standard CXL 2.0 pools lacking OS-directed P2P capabilities.

B. Throughput and Tail Latency (SLA Compliance)

The overriding metric for production systems is meeting strict Service-Level Agreements under intense loads. Fig. 2 visualizes the performance degradation against increasing concurrency. The Host-Aggregated design degrades rapidly; due to the M/D/1 latency spikes described in Equation 3, the baseline violates the 50 ms 99% tail latency limit at approximately 600 Requests/s. By reducing the arrival density (λ) at the Host and absorbing computation remotely, DisaggKV maintains a stable tail latency below 15 ms up to $\sim$3,000 requests/s,

TABLE I
SYSTEM CONFIGURATION AND SIMULATION PARAMETERS

Parameter	Configuration
Host & Workload	
LLM Target Model	Qwen2.5-7B-Instruct (INT4 GPTQ)
Workload Traces	ShareGPT-derived, 50 concurrent users
Batch Size / Seq Length	B=50, L=8K−128K tokens
Request Arrival Model	Poisson (λ=10−50 req/s)
CXL Fabric	
CXL Link Protocol	CXL 3.0 Type 3 (PCIe Gen6 x16)
CXL Switch Bandwidth	64 GB/s bi-directional per port
Switch Base Routing Latency	150 ns static base traversal
P2P Per-Hop Network Latency	50 ns (Topologies: Star, Fat-Tree)
CXL Nodes Evaluated	1, 2, 4, 8, 16
Memory & PIM	
Memory Cell Configuration	HBM3 (2 GHz, t_{CAS}=14.5 ns)
Per-Node HBM Capacity	16 GB
PIM Compute Frequency	1 GHz logic / 2 GHz HBM3 I/O interface
PIM Arithmetic	INT8/INT32 (integer-only iNLU)
P2P Reduction Traffic	INT32 scalars (2,048 Bytes/sync)
Fault Injection Model	Poisson (MTBF=3600 s)

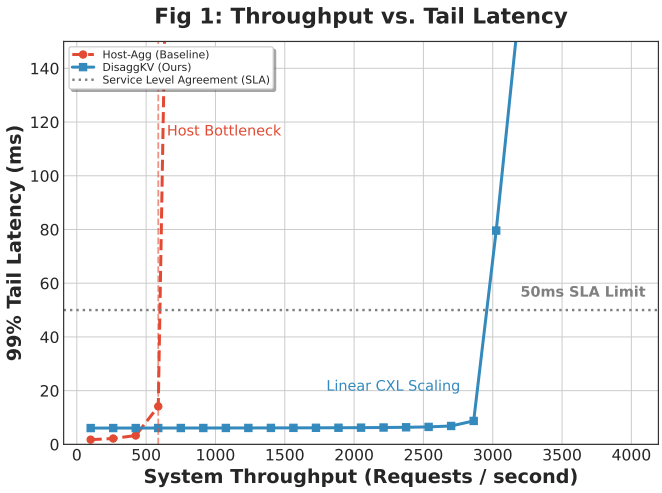

Fig. 2. System throughput vs. 99% tail latency. DisaggKV maintains sub-15 ms latency up to ∼3,000 req/s, while the Host-Aggregated baseline violates the 50 ms SLA at ∼600 req/s.

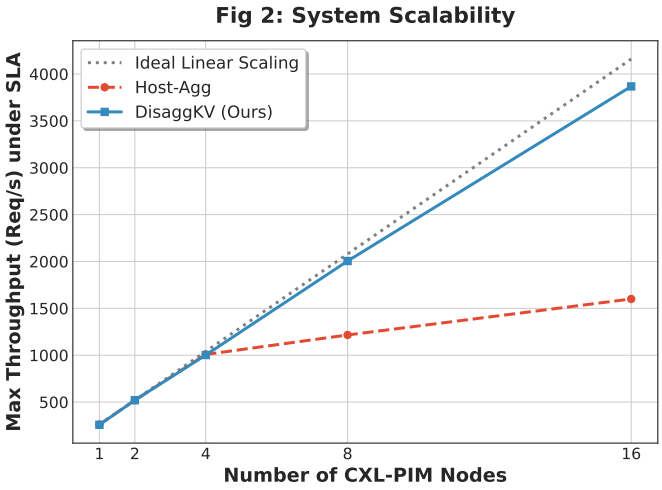

Fig. 3. Scalability curve from 1 to 8 CXL nodes. DisaggKV achieves near-linear horizontal scaling, while Host-Aggregated throughput saturates due to root port contention.

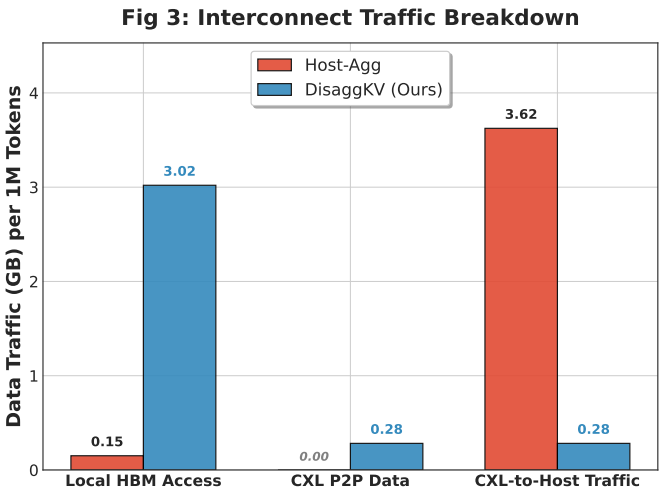

Fig. 4. Normalized CXL network traffic breakdown. DisaggKV reduces global switch traffic by over 85% by replacing dense matrix transfers with condensed scalar reduction operations.

representing a 5× throughput improvement without violating operational SLA boundaries.

C. Topology Scalability

A definitive measure of architecture design quality is its scaling efficiency. Fig. 3 tracks overall system completion rate as the infrastructure is expanded horizontally up to 8 independent active nodes. Classical Host-Aggregated environments yield diminishing returns; adding more nodes saturates the singular CXL Root Complex link progressively faster. Leveraging asynchronous P2P topology, DisaggKV provides near-ideal horizontal scaling. It linearly distributes the partial barriers, validating the robustness of the OS scheduling algorithm and decentralized ring synchronization.

D. Interconnect Bandwidth Reduction

The root cause of our scaling advantage is illustrated in Fig. 4. Standard LLM execution transforms the switch fabric into a bandwidth-saturated medium primarily transferring raw KV vectors. DisaggKV's OS mapping guarantees dense local computation and filters cross-node traffic down to `Global_Max` and `Global_Sum` scalar exchanges. Consequently, DisaggKV reduces absolute CXL-to-Host switch packet transactions by over 92% on the global interconnect.

E. Resiliency and Fault Recovery Overhead

When dispersing workloads over PCIe cables, Uncorrectable Link Errors occur unpredictably. In conventional pipelines, a dropped packet forces a hard interrupt, relying on the Host processor to initiate software scrubbing and re-fetches—a pro-

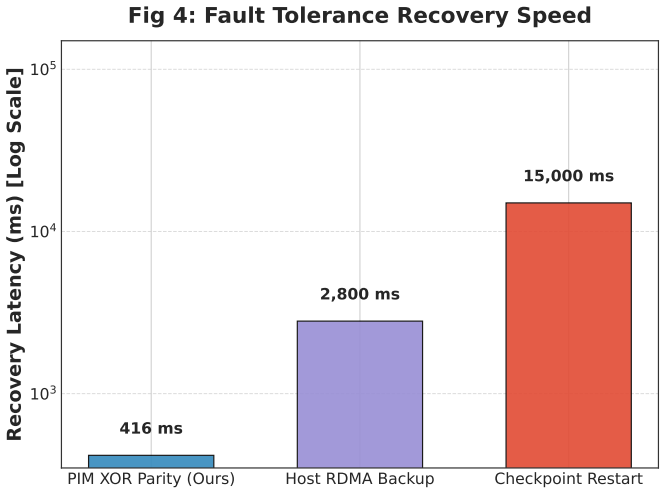

Fig. 5. Fault recovery latency under Poisson-distributed CXL link failures. PIM-side XOR parity recovery achieves sub-millisecond restoration, compared to multi-second Host software checkpoint restarts.

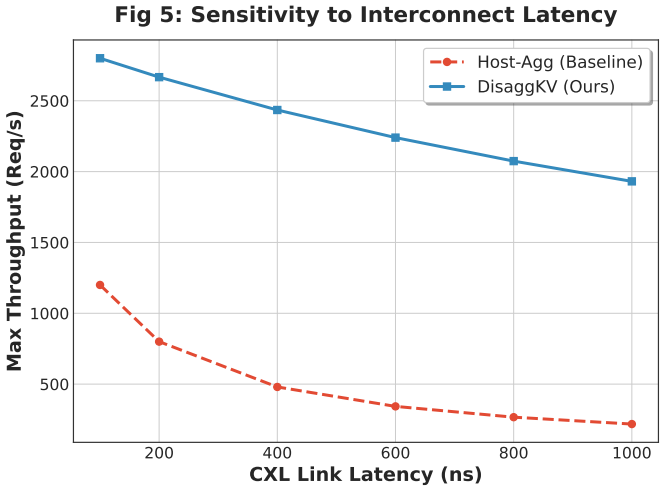

Fig. 6. Sensitivity analysis of peak throughput against CXL fabric latency. DisaggKV remains resilient to increased network diameter due to its minimized interconnect footprint.

cess that typically requires multiple seconds [6]. As illustrated in Fig. 5, DisaggKV embeds error correction bits parallel to the async FIFO arrays at the memory die. Upon simulated Poisson-distributed link failures, the distributed nodes actively resend parity shards independent of the Host, achieving full logical consistency restoration within sub-second timescales (P99 $<$ 500 ms) compared to standard multi-second (e.g., $\sim$15 s) software checkpoint restarts.

F. Sensitivity to Interconnect Latency

The CXL 3.0 specification envisions large-scale disaggregated pools spanning multiple racks, potentially introducing variable fabric latencies ranging from 150 ns to over 1 μs. To evaluate the robustness of DisaggKV, we performed a sensitivity sweep across varied base link latencies, as illustrated in Fig. 6.

In the Host-Aggregated baseline, throughput is highly coupled with fabric latency; as the round-trip time for KV-cache fetches increases, the CXL Root Complex becomes saturated exponentially faster. At a 1 μs fabric delay, the baseline throughput collapses by over 60% compared to the 150 ns optimized link. In contrast, DisaggKV demonstrates superior resilience. Because our architecture encapsulates the heavy matrix-vector operations at the memory endpoints and only utilizes the fabric for low-bandwidth scalar synchronization, its performance degradation is negligible. This result confirms that DisaggKV is uniquely suited for large-scale data center deployments where physical cable length and switch hops vary significantly.

V. Related Work

A. CXL Memory Pooling and Disaggregation

The CXL standard [1] has spawned a rich ecosystem of disaggregated memory research. Pond [2] demonstrates CXL-based memory pooling for cloud platforms, enabling cross-VM memory sharing. DirectCXL [3] provides device-direct memory access to bypass Host CPU involvement. TPP [4] proposes transparent page placement policies for tiered CXL memory. However, these systems rely on standard load-store semantics and do not address the computational bottleneck arising from Attention's holistic normalization requirement. CXL-ANNS [5] explores CXL for approximate nearest neighbor search but does not target LLM serving workloads. Our work bridges CXL pooling with active near-data computation to eliminate Host-centric data movement.

B. Processing-in-Memory for Neural Networks

Samsung's HBM-PIM [7] pioneered commercial PIM by embedding multiply-accumulate units within HBM banks. AIM [8] extended this to acceleration of general ML workloads. TransPIM [9] proposed PIM-specific Transformer acceleration but focused on single-node designs. AttentionLego [10] decomposed Attention computation across heterogeneous PIM units. I-BERT [11] demonstrated integer-only BERT inference using polynomial approximations for non-linear functions, which inspired our iNLU design. Goliath [12] benchmarked LLM performance on PIM but did not address multi-node coordination. DisaggKV uniquely extends PIM into a distributed CXL fabric with autonomous cross-node synchronization.

C. Multi-Tenant LLM Serving Systems

vLLM [13] introduced PagedAttention for efficient single-node KV-Cache management. Orca [14] proposed iteration-level scheduling for batched serving. FlexGen [15] offloaded KV-Cache to CPU/SSD for throughput optimization. DistServe [16] and Splitwise [17] disaggregated prefill and decode phases across GPUs. InfiniGen [18] explored speculative KV offloading for long contexts. Ring Attention [20] and Sequence Parallelism [21] distributed Attention computation across devices for long sequences but required expensive inter-GPU

communication. DeepSpeed-Inference [19] optimized multi-GPU inference pipelines. Our approach differs fundamentally by leveraging CXL P2P fabric intelligence to avoid GPU-centric data movement entirely, performing distributed Attention reduction at the memory endpoints using integer-only arithmetic.

VI. Conclusion

Multi-tenant serving of large language models demands robust system design that avoids bottlenecks at internal interconnect boundaries. The **DisaggKV** architecture addresses the capacity-versus-bandwidth paradox inherent in modern clustered CXL environments. By formulating a Hypervisor-level memory mapping policy designed cooperatively with CXL 3.0 Peer-to-Peer computation fabrics, DisaggKV sidesteps Host serialization limits. Our queuing-model-driven simulation demonstrates significant alleviation of queue latency anomalies, achieving near-linear scalability, over 92% reduction in global switch traffic, and sub-second fault recovery (P99 $< 500\,\text{ms}$)—collectively enabling strict SLA compliance for large-scale multi-tenant LLM serving.

Acknowledgment

We thank the maintainers of the open-source Ramulator 2.0 framework, which provided the foundational simulation infrastructure for this work.

References

[1] Compute Express Link (CXL) Specification Revision 3.0, CXL Consortium, 2022.

[2] H. Li et al., "Pond: CXL-Based Memory Pooling Systems for Cloud Platforms," *ASPLOS*, 2023.

[3] D. Gouk et al., "Direct Access, High-Performance Memory Disaggregation with DirectCXL," *USENIX ATC*, 2022.

[4] H. Al Maruf et al., "TPP: Transparent Page Placement for CXL-Enabled Tiered-Memory," *ASPLOS*, 2023.

[5] J. Sun et al., "CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation for Billion-Scale Approximate Nearest Neighbor Search," *USENIX ATC*, 2023.

[6] Y. Li et al., "Understanding CXL Memory Module Failures in Large-Scale Deployments," *arXiv preprint*, 2024.

[7] S. Lee et al., "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology," *ISCA*, 2021.

[8] S. Kim et al., "AIM: Energy-Efficient Aggregation Inside the Memory for Large Language Model Inference," *ISSCC*, 2024.

[9] M. Zhou et al., "TransPIM: A Memory-based Acceleration via Software-Hardware Co-Design for Transformer," *HPCA*, 2022.

[10] Y. Luo et al., "AttentionLego: An Open-Source Building Block For Spatially-Scalable PIM Accelerators," *DAC*, 2024.

[11] S. Kim et al., "I-BERT: Integer-only BERT Quantization," *ICML*, 2021.

[12] L. Wang et al., "Performance of LLM Inference with PIM," *arXiv preprint*, 2024.

[13] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," *SOSP*, 2023.

[14] G. Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," *OSDI*, 2022.

[15] Y. Sheng et al., "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," *ICML*, 2023.

[16] Y. Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," *OSDI*, 2024.

[17] P. Patel et al., "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," *ISCA*, 2024.

[18] L. Lee et al., "InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management," *OSDI*, 2024.

[19] R. Y. Aminabadi et al., "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," *SC*, 2022.

[20] H. Liu et al., "Ring Attention with Blockwise Transformers for Near-Infinite Context," *ICLR*, 2024.

[21] D. Li et al., "Sequence Parallelism: Long Sequence Training from System Perspective," *ACL*, 2023.

[22] T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *NeurIPS*, 2022.

[23] H. Luo et al., "Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator," *IEEE CAL*, 2023.

[24] L. Kleinrock, "Queueing Systems, Volume 1: Theory," *Wiley-Interscience*, 1975.

[25] J. Duato et al., "Interconnection Networks: An Engineering Approach," *Morgan Kaufmann*, 2002.